Universidad Tecnológica Nacional — Facultad Regional Buenos Aires

Departamento de Ingeniería en Sistemas de Información — Diseño de Sistemas (2026)

Arquitectura de la Plataforma DonaTrack

Memoria Técnica, Persistencia Políglota y Patrones de Diseño Arquitectónico (Entrega 5)

Equipo de Desarrollo e Ingeniería de Software — **DonaTrack (Grupo 7)**

Buenos Aires, Argentina — Octubre de 2026

Resumen Ejecutivo

En el presente documento exponemos la fundamentación formal y las decisiones de diseño arquitectónico que nosotros, como equipo de ingeniería, adoptamos para la plataforma **DonaTrack** en el marco de la Entrega 5 de Diseño de Sistemas. Se detalla la descomposición en microservicios lógicos desacoplados gobernados por el patrón de **Arquitectura Hexagonal** (Puertos y Adaptadores), garantizando el aislamiento absoluto del Dominio frente a tecnologías de infraestructura. Se defiende y justifica la estrategia de **persistencia políglota**, evaluando cuantitativa y cualitativamente los *trade-offs* de la migración del subsistema de Logística a una base documental en **MongoDB 7.0** frente al esquema relacional normalizado en **PostgreSQL 16** empleado por Donaciones, Incentivos y Notificaciones. Finalmente, se abordan los patrones de integración asíncrona mediante **RabbitMQ** y el diseño de la **Capa Anticorrupción (ACL)** con enrutamiento tolerante a fallos ante contingencias en infraestructura *cloud*.

Palabras clave: *Arquitectura Hexagonal, Persistencia Políglota, MongoDB, PostgreSQL, Arquitectura Dirigida por Eventos, RabbitMQ, Bounded Contexts, Tolerancia a Fallos, Domain-Driven Design.*

En este documento presentamos las decisiones arquitectónicas que nosotros, como equipo de desarrollo, tomamos para el diseño e implementación de la plataforma **DonaTrack**. A lo largo de esta quinta entrega de la cátedra de Diseño de Sistemas, nos propusimos consolidar una plataforma robusta, altamente desacoplada, escalable y mantenible.

A continuación, detallamos no solo las tecnologías elegidas, sino fundamentalmente los motivos, los *trade-offs* que evaluamos en cada encrucijada de diseño y las soluciones de ingeniería que construimos para garantizar que DonaTrack responda a las necesidades reales de donantes, transportistas y organizaciones comunitarias.

1. Estilo Arquitectónico Principal: Microservicios Lógicos y Arquitectura Hexagonal

1.1. De la Visión Monolítica a Microservicios Desacoplados (SOA)

Al proyectar la solución, evaluamos inicialmente implementar una aplicación monolítica tradicional por su simplicidad inicial de despliegue. Sin embargo, rápidamente identificamos que los ritmos de cambio y las exigencias de escalabilidad de cada área del sistema eran completamente dispares:

- El módulo de **Donaciones** maneja el catálogo de necesidades y la intención de los donantes.
- El módulo de **Logística** experimenta picos de lectura intensa y actualizaciones geoespaciales frecuentes por parte de choferes en calle.

- Los módulos de **Incentivos** y **Notificaciones** operan reactivamente en respuesta a eventos de negocio.

Por este motivo, nosotros decidimos implementar una **Arquitectura Orientada a Servicios (SOA)**, materializada mediante microservicios lógicos autónomos (`donatrack-donaciones`, `donatrack-logistica`, `donatrack-incentivos` y `donatrack-notificaciones`). Cada uno de estos servicios cuenta con su propio ciclo de vida, configuración independiente y un modelo de datos encapsulado que ningún otro servicio puede consultar de manera directa por base de datos.

1.2. Arquitectura Hexagonal (Puertos y Adaptadores)

Para estructurar internamente cada uno de nuestros microservicios, adoptamos de forma estricta el patrón de **Arquitectura Hexagonal (Puertos y Adaptadores)**. Nuestro objetivo principal fue aislar las reglas y entidades del negocio de cualquier detalle tecnológico o framework externo (como Spring Boot, Hibernate, Spring Data MongoDB o librerías AMQP).

Organizamos cada microservicio en tres capas concéntricas con responsabilidades bien delimitadas:

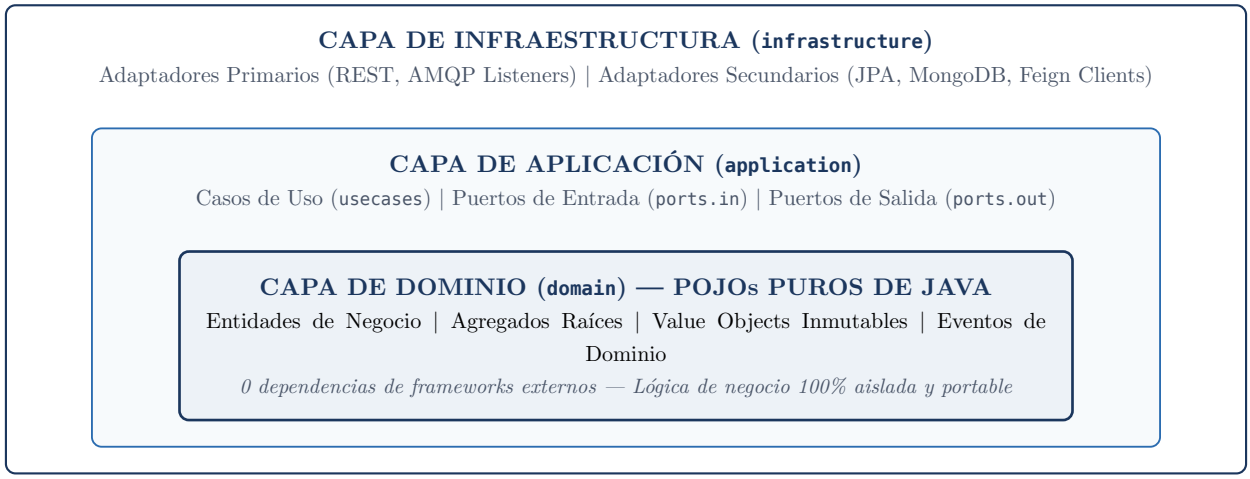


Figura 1: Estructura concéntrica de capas en la Arquitectura Hexagonal y aislamiento del Dominio.

- **Capa de Dominio (`domain`):** Contiene el corazón del negocio: entidades de dominio, agregados raíces, *Value Objects* y eventos de dominio. Nos impusimos la regla de que esta capa esté compuesta exclusivamente por **POJOs puros de Java**. No permitimos ninguna anotación de frameworks (ni de Spring, ni `@Entity` de JPA, ni `@Document` de MongoDB). De esta forma, si el día de mañana decidimos cambiar de framework o de motor de base de datos, el núcleo de DonaTrack permanece 100% inalterado.
- **Capa de Aplicación (`application`):** Aquí residen los casos de uso (`usecases`), responsables de orquestar la ejecución de las operaciones de negocio, coordinar transacciones y publicar eventos. La comunicación con el exterior se define mediante contratos rigurosos:
  - *Puertos de Entrada (`ports.in`):* Interfaces que declaran qué operaciones ofrece el sistema al exterior (por ejemplo, `IniciarRutaUseCase`, `RegistrarDonacionUseCase`).
  - *Puertos de Salida (`ports.out`):* Interfaces que especifican qué servicios necesita la aplicación desde la infraestructura (por ejemplo, `RutaRepartoRepositoryPort`, `EventPublisherPort`, `GeocodingClientPort`).
- **Capa de Infraestructura (`infrastructure`):** Es la capa periférica donde implementamos los detalles tecnológicos concretos:
  - *Adaptadores Primarios o de Entrada (`adapters.in`):* Controladores REST que transforman peticiones HTTP JSON en llamadas a los casos de uso, y listeners de RabbitMQ que escuchan mensajes de colas.

- *Adaptadores Secundarios o de Salida (`adapters.out`):* Clases que implementan los puertos de salida, tales como repositorios basados en Spring Data JPA o Spring Data MongoDB, clientes HTTP con OpenFeign y publicadores de mensajes AMQP.
- *Configuración (`config`):* Clases de configuración de Spring Boot, conversores de tipos para la base de datos y definiciones de beans.

¿Qué beneficio obtuvimos con esta decisión? Pudimos escribir pruebas unitarias exhaustivas sobre el dominio y los casos de uso utilizando *mocks* para los puertos de salida, sin necesidad de levantar bases de datos en memoria, contenedores Docker ni el contexto pesado de Spring Boot. La velocidad de ejecución de nuestra suite de tests es instantánea.

2. Estrategia de Persistencia Políglota y Bases de Datos (Entrega 5)

En esta quinta entrega, nos enfrentamos a un requerimiento fundamental: romper con el paradigma de «un único motor relacional para todo el sistema» y adoptar una **Estrategia de Persistencia Políglota**. Nuestro objetivo fue analizar la naturaleza operativa de cada *Bounded Context* y seleccionar para cada uno el motor y el modelo de datos que maximice su eficiencia y naturalidad de representación.

A continuación, resumimos en una matriz cómo distribuimos la persistencia en DonaTrack:

Microservicio	Motor y Paradigma	Esquema / Base	Propósito Principal
`donaciones`	PostgreSQL 16 (Relacional)	`donaciones`	Integridad ACID en bienes y donantes
`incentivos`	PostgreSQL 16 (Relacional)	`incentivos`	Trazabilidad y cómputo de puntos
`notificaciones`	PostgreSQL 16 (Relacional)	`notificaciones`	Auditoría de envíos y canales
`logistica`	MongoDB 7.0 (Documental)	`donatrack_logistica`	Rutas jerárquicas sin JOINs

Tabla 1: Matriz de persistencia políglota por Bounded Context.

2.1. Persistencia Documental en Logística (MongoDB 7.0)

El cambio más profundo y desafiante que nosotros encaramos en esta entrega fue la migración completa del subsistema de Logística desde un esquema relacional tradicional en PostgreSQL hacia una base documental en **MongoDB 7.0**.

A. Justificación Técnica del Cambio de Motor (El Problema del JOIN Relacional)

Durante el análisis de las operaciones del chofer y el coordinador de transporte, notamos que el concepto central de este contexto es la `RutaDeReparto`. Una ruta de reparto es un agregado que agrupa un itinerario completo de paradas ordenadas cronológicamente, cada una con sus entregas o retiros asociados, las donaciones correspondientes, sus coordenadas geográficas, direcciones y los comprobantes de entrega registrados por el chofer.

En nuestro diseño relacional previo, recuperar la información completa de una ruta para que el chofer pudiera visualizar su itinerario diario en la aplicación móvil demandaba ejecutar una consulta con `JOIN` sobre 6 tablas normalizadas:

```
-- La pesadilla relacional que sufríamos en cada consulta del chofer:
SELECT * FROM rutas_reparto r
JOIN paradas p ON p.ruta_id = r.id
```

```
JOIN entregas e ON e.parada_id = p.id
LEFT JOIN comprobantes c ON c.entrega_id = e.id
JOIN direcciones d ON d.id = p.direccion_id
JOIN coordenadas coord ON coord.id = d.coordenada_id
WHERE r.id = ? ORDER BY p.orden ASC;
```

Esta consulta implicaba:

- Múltiples escaneos de índices y saltos de punteros en memoria para correlacionar claves foráneas.
- Un costo de CPU y tiempo de respuesta elevado ante solicitudes concurrentes de múltiples choferes en calle.
- Un mapeo relacional-objeto extremadamente complejo con riesgo de consultas redundantes (el problema de $N + 1$ `SELECT`).

Al analizar este escenario desde la perspectiva de *Domain-Driven Design*, comprendimos que `RutaDeReparto` es el **Agregado Raíz** por excelencia. Una parada o una entrega no tienen sentido de existencia fuera del contexto de su ruta contenedora. Por lo tanto, decidimos almacenar todo el árbol del agregado en un único documento BSON en MongoDB.

Con este enfoque documental:

- Recuperar la ruta completa del chofer se resuelve mediante una única lectura directa por clave primaria `_id`, con complejidad $O(1)$.
- Toda la información jerárquica (paradas, entregas, estados y coordenadas) viaja junta en el documento BSON, eliminando por completo la necesidad de *joins* en el motor de base de datos.

B. Estrategia de Desnormalización y Snapshot Histórico Inmutable

Uno de los dilemas más interesantes que tuvimos que resolver fue cómo relacionar el Camión y el Chofer con la Ruta de Reparto:

1. ¿Debíamos almacenar un identificador o una referencia viva (`DBRef`) apuntando a las colecciones de `camiones` y `choferes`?
2. ¿O debíamos embeber la información directamente dentro de la ruta?

Nosotros decidimos adoptar una estrategia de **Snapshot Histórico Inmutable**. Cuando un coordinador planifica y despacha una ruta, el sistema captura una fotografía exacta de los datos operativos del camión (patente, capacidad en kg, altura y volumen en m^3) y del chofer (nombre, legajo) y los almacena desnormalizados dentro del documento de la ruta.

¿Por qué tomamos esta decisión? Por una estricta razón de integridad histórica y legal. En el mundo real del transporte:

- Un chofer puede cambiar de número de teléfono, renovar su legajo o desvincularse de la organización el mes siguiente.
- Un camión puede ser reemplazado o cambiar su patente. Si hubiésemos utilizado referencias vivas, al consultar una ruta ejecutada hace seis meses veríamos los datos actuales del chofer y no los datos reales con los que el vehículo salió a la calle en esa fecha. Nuestro *snapshot* garantiza una auditoría inmutable e incontrovertible de cada operativo logístico.

C. Optimización de Lectura: Índice Compuesto Multikey B-Tree

Aunque el acceso por ID de ruta es $O(1)$, nos encontramos con una consulta de negocio crítica: la trazabilidad del donante. Cuando un donante ingresa a la aplicación y consulta el estado de su donación mediante el endpoint `GET /api/entregas/{idDonacion}/estado`, el sistema debe responder en qué ruta viaja dicha donación, qué chofer la transporta y cuál es el estado estimado de la entrega.

Dado que las entregas están contenidas dentro de un array de paradas (`paradas.entregas`), una búsqueda sin índices obligaría a MongoDB a realizar un escaneo secuencial (*collection scan*) sobre todos los documentos de rutas y desempaquetar cada uno de sus arrays internos, resultando en una complejidad inaceptable de $O(N \cdot M)$.

Para resolver esto de forma óptima, nosotros diseñamos y declaramos un **índice compuesto multikey B-Tree** en la colección `rutas_reparto`:

```
@CompoundIndex(
    name = "idx_rutas_entregas_donacion",
    def = "{'paradas.entregas.idEntrega': 1}"
)
```

Gracias a este índice *multikey*, MongoDB indexa cada elemento interno del array de entregas en una estructura de árbol B, permitiendo que la búsqueda de la ruta contenedora a partir del identificador de la donación se ejecute en tiempo logarítmico $O(\log N)$.

D. Validación Cuantitativa del Límite de 16 MB de BSON

Una de las restricciones arquitectónicas que impone MongoDB es que ningún documento BSON puede superar los 16 MB. Si nuestro documento de ruta creciera indefinidamente (el conocido antipatrón de *unbounded arrays*), el sistema colapsaría al intentar persistir rutas extensas.

Nosotros realizamos un cálculo de dimensionamiento riguroso basado en la operatoria de DonaTrack:

- Una ruta de reparto operativa en una jornada habitual contempla entre 15 y 50 paradas.
- En cada parada se gestionan en promedio entre 1 y 2 entregas o retiros.
- Una ruta de tamaño excepcional (por ejemplo, ante catástrofes o colectas masivas) no supera las 100 entregas.
- **Decisión de diseño clave sobre fotos y comprobantes:** Determinamos que las imágenes de los comprobantes y firmas que toma el chofer **nunca** se almacenan en Base64 dentro del documento BSON. Las fotos se suben a un almacenamiento de objetos externo y en el documento de MongoDB únicamente persistimos URLs públicas y metadatos de texto livianos (≈ 120 bytes por comprobante).

Resultado del cálculo métrico: Un documento de ruta con 100 entregas, 50 paradas, comprobantes, coordenadas y *snapshots* de chofer y vehículo promedia un peso de ≈ 150 KB:

$$\frac{150 \text{ KB}}{16.384 \text{ KB}} \approx 0,91\%$$

El documento ocupa **menos del 1%** del límite máximo de 16 MB permitido por MongoDB. Esto demuestra que nuestra decisión de embeber las paradas y entregas dentro de la ruta es completamente segura y no presenta riesgos de desbordamiento de memoria.

E. Mapeo BSON Binary ⇆ Java UUID (`MongoConfig.java`)

Durante la integración entre Spring Data MongoDB y el cluster de base de datos, nos encontramos con una incompatibilidad técnica: Spring Data serializa por defecto los `java.util.UUID` de Java como tipo `Binary` (Subtipo 4) en BSON. Si bien esto es eficiente a nivel binario, al consultar la base desde herramientas administrativas externas como `mongosh` o Compass, o al correr scripts de inicialización, los identificadores resultaban difíciles de inspeccionar o sufrían discrepancias en el orden de bytes (*endianness*).

Para resolver esto de raíz de forma elegante y centralizada, nosotros implementamos conversores personalizados registrados en nuestra clase de configuración `MongoConfig.java`:

- `BinaryToUuidConverter` (`@ReadingConverter`): Toma los 16 bytes crudos del BSON Binary y reconstruye con precisión el objeto `java.util.UUID`.
- `UuidToBinaryConverter` (`@WritingConverter`): Convierte el UUID de Java al formato binario estándar de MongoDB al persistir.

Con estos dos conversores registrados en el `MongoCustomConversions` de Spring Boot, garantizamos una persistencia transparente, consistente y totalmente compatible entre el código Java y las consultas nativas en consola.

F. Idempotencia y Doble Mecanismo de Inicialización de Datos

Para garantizar que cualquier miembro de nuestro equipo, los docentes o los evaluadores puedan levantar y probar el sistema inmediatamente sin pasos manuales, implementamos un mecanismo de inicialización de datos con garantías de idempotencia (es decir, que al reiniciar el servidor no se dupliquen camiones, choferes ni rutas):

1. **Inicialización en Docker Local (`database/mongo-init.js`):** Al levantar el contenedor `mongo-donatrack` mediante Docker Compose, este script nativo de JavaScript precarga colecciones protegiendo cada inserción con validaciones condicionales e idempotentes:

```
if (db.camiones.countDocuments() === 0) {
    db.camiones.insertMany([ ... ]);
}
```

2. **Inicialización en la Nube (`MongoDataInitializer.java`):** Dado que en entornos de nube como MongoDB Atlas no se ejecutan los scripts de arranque de Docker, desarrollamos un componente Spring Boot que implementa `CommandLineRunner`. Al encenderse la aplicación, consulta mediante los repositorios si existen datos precargados. Si la base se encuentra vacía, inyecta automáticamente a través de los casos de uso un lote inicial consistente de camiones, choferes, ítems pendientes de recolección y rutas de prueba.

G. Preservación Estricta del Aislamiento Hexagonal

Queremos resaltar un logro de ingeniería del cual nos sentimos especialmente satisfechos: **la migración de Logística a MongoDB no requirió modificar ni una sola línea de código en nuestras clases de dominio**.

Nuestras entidades de negocio (`RutaDeReparto`, `ParadaDeRuta`, `Entrega`, `Coordenada`, `Direccion`) permanecieron completamente agnósticas a la base de datos. Toda la adaptación a MongoDB se contuvo estrictamente dentro de la capa de infraestructura:

- Creamos documentos específicos de persistencia (`RutaDeRepartoDocument`, `CamionDocument`, `ChoferDocument`) anotados con `@Document`, `@Id` y `@Field`.
- Desarrollamos mappers bidireccionales (`RutaDeRepartoMongoMapper`) encargados de transformar las entidades puras del dominio en documentos de MongoDB al guardar, y viceversa al recuperar los datos.

Si en el futuro la organización decidiera volver a una base relacional o adoptar una base de grafos como Neo4j para optimizar caminos, nuestro dominio permanecería 100% intacto.

2.2. Persistencia Relacional en Donaciones, Incentivos y Notificaciones (PostgreSQL 16)

Mientras que en Logística adoptamos el paradigma NoSQL, para los microservicios de Donaciones, Incentivos y Notificaciones decidimos mantener el motor relacional **PostgreSQL 16**, dado que sus modelos se benefician enormemente de las garantías de consistencia transaccional ACID y la normalización de entidades.

A. Segregación por Esquemas Lógicos (`donaciones`, `incentivos`, `notificaciones`)

Al plantear la arquitectura en la nube (utilizando Supabase), nos encontramos ante dos caminos posibles:

1. Aprovisionar tres instancias de bases de datos PostgreSQL físicas e independientes (una por microservicio).
2. Utilizar una única instancia física de PostgreSQL y crear dentro de ella tres esquemas lógicos segregados (`CREATE SCHEMA donaciones`, `CREATE SCHEMA incentivos`, `CREATE SCHEMA notificaciones`).

Nosotros decidimos optar por la **segregación por esquemas lógicos**. Aprovisionar tres bases de datos administradas independientes en proveedores *cloud* generaba un sobrecosto económico innecesario y multiplicaba las tareas de mantenimiento y monitoreo. La segregación por esquemas nos brinda

exactamente el mismo nivel de aislamiento lógico y seguridad: cada microservicio se conecta con credenciales y un `currentSchema` restringido a sus propias tablas, impidiendo cualquier acoplamiento cruzado o consultas que vulneren los límites del *Bounded Context*, pero optimizando los costos de infraestructura al compartir el servidor físico.

B. Generación de Identidad en el Dominio vs. Secuencias Autoincrementales

En lugar de utilizar el patrón clásico de identificadores numéricos autoincrementales (`Long id` con `@GeneratedValue(strategy = GenerationType.IDENTITY)`), nosotros decidimos que todas las entidades del sistema generen su propio identificador como un **UUID** en el momento de su instanciación en el Dominio (`UUID.randomUUID()`).

¿Por qué evitamos los IDs autoincrementales de base de datos?

- Con `@GeneratedValue`, una entidad no adquiere identidad hasta que se ejecuta un `flush` o `commit` contra la base de datos relacional. Esto rompe la autonomía del dominio e impide generar y publicar Eventos de Dominio válidos antes de la persistencia.
- Con UUIDs generados en el dominio, las entidades nacen con identidad propia e inmutable. Además, esto facilita la integración asíncrona entre microservicios, ya que los identificadores pueden crearse en el cliente o en el servicio emisor sin riesgo de colisión entre distintas bases de datos.

C. Estrategias de Mapeo de Herencia JPA

En el modelado de nuestras entidades en PostgreSQL, nos enfrentamos a dos jerarquías de clases con naturalezas muy distintas, y decidimos aplicar estrategias de herencia JPA diferenciadas según el balance de *trade-offs* de cada caso:

- **Estrategia `InheritanceType.JOINED` para Personas y Roles:** La jerarquía de personas (`Persona` → `PersonaHumana`, `PersonaJuridica`) y roles (`Rol` → `RolDonante`, `RolBeneficiario`, `RolRepresentante`) involucra atributos muy específicos y de gran peso legal (por ejemplo, CUIT, razón social y datos fiscales en personas jurídicas versus DNI y fecha de nacimiento en personas humanas). Elegimos `JOINED`: aunque penaliza ligeramente con un `JOIN` en consultas polimórficas, garantiza una normalización de base de datos estricta, evita tener tablas inmensas con decenas de columnas con valores `NULL` y permite definir restricciones de integridad referencial (`NOT NULL`, `CHECK`, claves foráneas) precisas para cada subtipo.
- **Estrategia `InheritanceType.SINGLE_TABLE` para Necesidades:** En el catálogo de necesidades, modelamos variantes como `NecesidadRecurrente` (solicitudes periódicas como viandas semanales) y `NecesidadExtraordinaria` (campañas urgentes por inundaciones o catástrofes). Al analizar sus atributos, descubrimos que ambas subclases comparten más del 90% de sus campos (título, descripción, cantidad solicitada, unidad de medida, estado, beneficiario asociado). Por lo tanto, elegimos `SINGLE_TABLE` con una columna discriminadora `dtype`. De esta manera, el catálogo de necesidades —que es consultado intensamente por donantes desde la interfaz web— se recupera a máxima velocidad mediante escaneos simples sin penalización de *joins*.

D. Value Objects Aplanados (Flattening / `@Embedded`)

En nuestro modelo de dominio existen conceptos de gran valor semántico que carecen de identidad propia e inmutable por sí mismos: `Contacto`, `DocumentoIdentidad`, `Coordenada`, `Provincia` y `Pais`. Nosotros decidimos modelarlos rigurosamente como *Value Objects* en el dominio y aplanarlos mediante `@Embedded` y `@Embeddable` de JPA dentro de las tablas de sus entidades contenedoras (`personas` y `direcciones`). Esto evita la proliferación de tablas auxiliares artificiales con claves foráneas accesorias, manteniendo el esquema de base de datos limpio y cohesionado.

3. Patrones de Integración y Diseño Guiado por el Dominio (DDD)

3.1. Bounded Contexts y Lenguaje Ubicuo

Cada uno de los cuatro microservicios de DonaTrack representa un **Bounded Context** (Contexto Delimitado) formal en términos de *Domain-Driven Design*. Cada contexto posee su propio **Lenguaje Ubicuo**, adaptado a su propósito operativo:

- Para el contexto de **Donaciones**, una «Donación» representa un acto solidario con implicancias de estado, catálogo de bienes ofrecidos, validaciones formales y compromiso del donante.
- Para el contexto de **Logística**, esa misma donación no se concibe como un acto solidario, sino como un `ItemDonacion`: una unidad física de carga con peso en kilogramos, volumen en metros cúbicos y un par de coordenadas geográficas de origen y destino que deben consolidarse en la bodega de un camión.
- Para el contexto de **Incentivos**, la donación completada se traduce simplemente en un evento generador de puntaje según las reglas de gamificación vigentes.

Ningún modelo conceptual contamina al otro; las transformaciones ocurren explícitamente en las fronteras de integración.

3.2. Arquitectura Dirigida por Eventos (EDA) con RabbitMQ

Al diseñar la comunicación entre microservicios, nos preguntamos: ¿debemos usar llamadas HTTP REST síncronas entre servicios o comunicación asíncrona por eventos?

Nosotros decidimos adoptar una **Arquitectura Dirigida por Eventos (Event-Driven Architecture - EDA)** utilizando **RabbitMQ** como *message broker* para desacoplar temporalmente los procesos. Si hubiésemos elegido llamadas REST síncronas:

- Cuando un chofer finalizara una entrega, el servicio de Logística tendría que invocar a Donaciones para cambiar el estado, y luego invocar a Incentivos para otorgar puntos, y luego a Notificaciones para enviar un WhatsApp.
- Si el servicio de Incentivos sufriera una caída momentánea, la llamada HTTP fallaría y la confirmación de la entrega del chofer se vería abortada o bloqueada.

Con nuestra arquitectura de eventos basada en RabbitMQ, la comunicación es completamente asíncrona y reactiva:

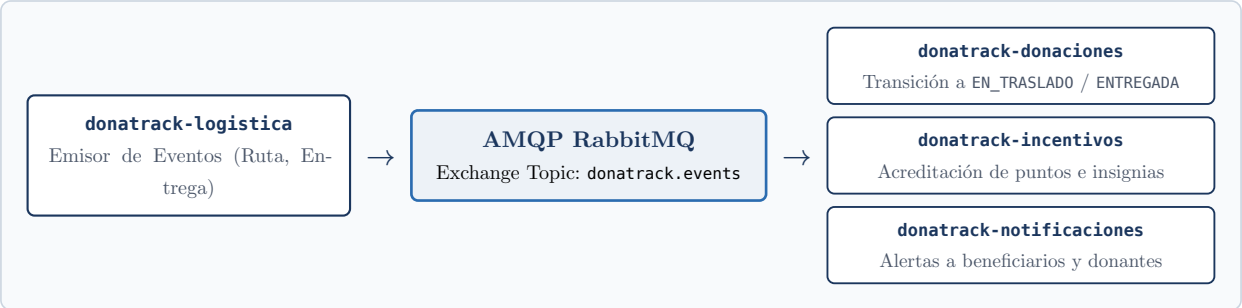


Figura 2: Topología de comunicación asíncrona orientada a eventos mediante RabbitMQ.

Diseñamos e implementamos los siguientes eventos de dominio clave:

- `RutaIniciadaEvent`: Emitido por Logística cuando el camión parte del depósito. Es consumido por Donaciones para transicionar automáticamente el estado de las donaciones asignadas a `EN_TRASLADO`.
- `EntregaRealizadaEvent`: Emitido por Logística cuando el chofer confirma una entrega exitosa con foto y firma. Es consumido por:
  - **Donaciones:** Marca la donación como `ENTREGADA` y registra la URL del comprobante.
  - **Incentivos:** Computa los puntos correspondientes para el donante y evalúa si amerita una nueva insignia.

- `EntregaFallidaEvent`: Emitido por Logística cuando el camión llega al destino pero el donante o beneficiario está ausente. Es consumido por:
  - **Donaciones:** Reabre el ítem para una reprogramación futura.
  - **Notificaciones:** Dispara una alerta inmediata por correo o mensaje al involucrado informando el inconveniente.

El Trade-Off: Nosotros decidimos abrazar conscientemente la **Consistencia Eventual** en lugar de transacciones distribuidas complejas de dos fases (*2PC - Two-Phase Commit*). En un sistema de impacto social y logística como DonaTrack, priorizamos la **Alta Disponibilidad** y el rendimiento: preferimos que los puntos de incentivo se acrediten con unos segundos de desfase eventual antes que bloquear los camiones en la calle esperando un *lock* distribuido en la base de datos.

3.3. Capa Anticorrupción (ACL) y Broker de Logística Resiliente

Durante el despliegue del sistema en la nube, nos encontramos con un problema real e ineludible de infraestructura: los servicios gratuitos de alojamiento (como Render) aplican políticas de suspensión por inactividad. Cuando un microservicio no recibe tráfico durante 15 minutos, su contenedor entra en hibernación (*cold-start*), demorando entre 30 y 50 segundos en responder la siguiente solicitud entrante, o respondiendo directamente con *timeouts*.

Si el microservicio `donatrack-donaciones` hiciera una llamada HTTP simple hacia `donatrack-logistica` para dar de alta un ítem donado, el usuario donante sufriría un error `504 Gateway Timeout` en la pantalla de su navegador.

Para blindar nuestro sistema contra esta falla, nosotros diseñamos e implementamos el patrón **Anti-corruption Layer (ACL)** materializado en la clase `LogisticaBrokerAdapter.java`. Implementamos un algoritmo de enrutamiento resiliente en 3 niveles con *fallback* automático:

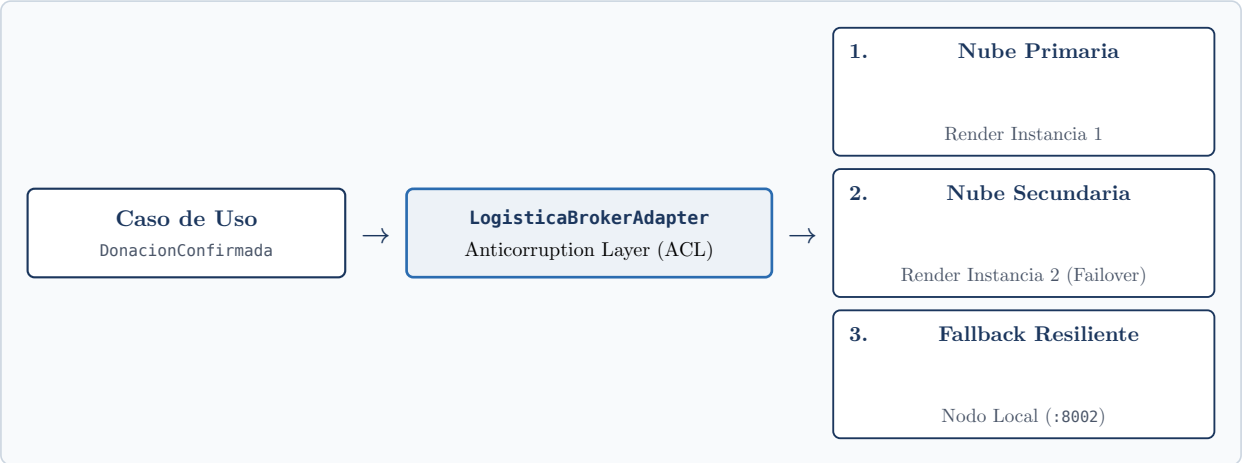


Figura 3: Algoritmo de conmutación por error en tres niveles (Anticorruption Layer - ACL) ante cold--starts de Render.

1. **Intento 1 (Nube Primaria):** Se intenta enviar la petición al endpoint primario desplegado en Render mediante un cliente Feign (`LogisticaRemoto1Client`).
2. **Intento 2 (Nube Secundaria / Reintento):** Si la instancia primaria está en proceso de arranque o no responde, el broker conmuta inmediatamente hacia la instancia de réplica secundaria (`LogisticaRemoto2Client`).
3. **Intento 3 (Fallback Local Resiliente):** Si ambos nodos en la nube no responden a tiempo, el broker deriva de manera transparente la operación hacia el nodo local de respaldo en `http://localhost:8002` (`LogisticaLocalClient`).

¿Qué logramos con esta arquitectura? Aislamos por completo el dominio de Donaciones de las fragilidades de la red y garantizamos una experiencia de usuario impecable: el donante siempre recibe

una confirmación inmediata de su donación, mientras que la capa de infraestructura resuelve el balanceo y la resiliencia en segundo plano.

4. Despliegue e Infraestructura (Deployment)

Nosotros diseñamos la infraestructura de DonaTrack bajo una premisa fundamental: **portabilidad absoluta y paridad entre entornos de desarrollo y producción**. A través del uso de perfiles de Spring Boot (`PROFILE=local` y `PROFILE=prod`), el sistema puede ejecutarse tanto en una laptop de desarrollo aislada sin conexión a internet como en una arquitectura distribuida en la nube.



Figura 4: Topología general de despliegue físico y persistencia políglota de DonaTrack.

4.1. Perfil Local (`PROFILE=local`)

Para el entorno local, configuramos una orquestación integral con Docker Compose que levanta los servicios requeridos dentro de una red privada virtual (`donatrack-network`):

- `postgres-donatrack`: Servidor PostgreSQL 16 en el puerto `5432` con volumen persistente `postgres_data`, que inicializa automáticamente los esquemas `donaciones`, `incentivos` y `notificaciones`.
- `mongo-donatrack`: Servidor NoSQL MongoDB 7.0 en el puerto `27017` con volumen persistente `mongo_data` y script `mongo-init.js` para precarga de camiones y choferes.
- `rabbitmq-donatrack`: Broker de mensajería RabbitMQ con consola de administración Management UI expuesta en el puerto `15672` y puerto de mensajería AMQP en el `5672`.
- `n8n-donatrack`: Plataforma de orquestación de flujos de trabajo y webhooks en el puerto `5678`.
- **Microservicios DonaTrack:** Ejecutándose en los puertos locales:
  - `donatrack-donaciones` → `8000`
  - `donatrack-incentivos` → `8001`
  - `donatrack-logistica` → `8002`
  - `donatrack-notificaciones` → `8003`

4.2. Perfil Cloud / Producción (`PROFILE=prod`)

Para el entorno productivo en la nube, seleccionamos proveedores de infraestructura como servicio (PaaS y DBaaS) que garantizan alta disponibilidad sin costos de administración de servidores dedicados:

- **Base de Datos Relacional Cloud:** Instancia de **Supabase PostgreSQL** (alojada en AWS región `us-east-1`), con conexiones seguras SSL (puerto `6543` / `5432`) que alberga los esquemas lógicos `donaciones`, `incentivos` y `notificaciones`.
- **Base de Datos Documental Cloud:** Cluster administrado en **MongoDB Atlas** (`donatrack.ofn4ag4.mongodb.net`) con réplicas distribuidas y autenticación segura para la base de datos `donatrack_logistica`.
- **Broker de Mensajería Cloud:** Cluster de **CloudAMQP** (`jackal.rmq.cloudamqp.com`) con protocolo seguro AMQPS, *virtual hosts* dedicados y colas durables.

- **Plataforma de Cómputo:** Despliegue de los microservicios en **Render**, con pipelines de integración y despliegue continuo (CI/CD) vinculados a nuestro repositorio de código, complementados con la resiliencia ante *cold-starts* provista por nuestro `LogisticaBrokerAdapter`.

5. Conclusión y Síntesis del Equipo

El diseño arquitectónico que implementamos para **DonaTrack** en esta Entrega 5 no es producto del azar ni de la adopción acrítica de tecnologías de moda. Cada una de las decisiones que tomamos —la separación en microservicios lógicos, la preservación del dominio puro mediante Arquitectura Hexagonal, la migración de Logística a MongoDB para resolver el problema de los *joins* masivos, la desnormalización inmutable de choferes y camiones, el índice *multikey* $O(\log N)$, la persistencia relacional con esquemas segregados en PostgreSQL y el desacoplamiento asíncrono con RabbitMQ— responde a un análisis metódico de *trade-offs* de ingeniería.

Logramos una arquitectura donde cada *Bounded Context* almacena sus datos en el motor que mejor refleja su naturaleza, el dominio permanece 100% aislado de la tecnología de persistencia y la infraestructura cuenta con mecanismos automáticos de resiliencia ante contingencias en la nube. Con esta base, DonaTrack queda preparado para escalar con solvencia y responder de manera confiable a la misión comunitaria para la cual fue concebido.